



Pong

▼ 1. Komponenten bauen



Grundidee: Arbeit in Turtle

- nicht nur eine Turtle sondern mehrere → jede ist eine Komponente

Komponenten

Screen (vorgegeben)

Variablenname/ Funktion	Nutzen	Wert	Anmerkung
width	Breite	600	anpassbar
height	Höhe	400	anpassbar
setup(width, height)	Screen initialisieren		x=0, y=0 ist die Mitte des Fensters
bgcolor	Hintergrundfarbe	white	"coolerer" Look: schwarz, dann müssen die Farben der Turtles auch angepasst werden
title	Titel des Fentsters	pong	irrelevant
tracer	Animation stoppen	0,0	essentiell für schnelle Bewegung!

1.1 Paddles

Variablenname/ Funktion	Nutzen	Wert	Anmerkung
p_width	Breite	100	anpassbar
p_length	Höhe	20	anpassbar
color	Farbe	black	
shape	Form	square	
shapeseize	Größe einstellen	p_length/20, p_width/20	wird im Verhältnis angegeben (1=20px)
penup	Turtle zieht keine Linien wenn sie sich bewegt		
goto	Paddle an Ausgangsposition positionieren/ bewegen	-width/2+width/12, 0	x und y sind die Mitte des Rechtecks

1.2 Ball

Variablenname/ Funktion	Nutzen	Wert	Anmerkung
b_radius	Radius	20	
shape	Form	circle	
color	Farbe	red	
shapeseize	Größe einstellen	b_radius/20	wird im Verhältnis angegeben (1=20px)
penup()	Turtle zieht keine Linien wenn sie sich bewegt		
goto	Ball an Ausgangsposition/ bewegen	0,0	x und y sind die Mitte des Rechtecks
dx	Änderungsrate x- Richtung	0	
dy	Änderungsrate y- Richtung	0	

1.3 Spielstand

Variablenname/ Funktion	Nutzen	Wert	Anmerkung
points_p1	Punkte Spieler 1	0	Ziel: 10P
points_p2	Punkte Spieler 2	0	Ziel: 10P
penup()			
hideturtle()	Turtle verstecken		Turtle soll hier etwas malen, nicht der Körper der Turtle stellt ein Spielobjekt dar
goto()	Positionierung	-width/6,height/4	muss/ kann noch verbessert werden
write()	Spielstand anzeigen	"Spieler 1: "+str(points_p1)+"", Spieler 2: "+str(points_p2)	
clear()	Anzeige erneuern/ löschen		wird noch nicht beim Setup gebraucht

▼ 2. Bewegung hinzufügen

2.1 Ball

- Bewegung dargestellt durch Verschiebung des Balls um gewisse Pixelanzahl
 - je höher die Werte zur Verschiebung, desto schneller bewegt sich der Ball
- Richtungen
 - nach links/rechts → x ändern
 - nach oben/ unten → y ändern
- für mehr Abwechslung: Änderungsrate zufällig generieren
 - ändert sich immer nach Punkt

2.2 Paddles

- Bewegung durch Tastendruck
 - P1: w, s
 - P2: up, down
- solange gedrückt nach oben/ unten bewegen
 - y erhöhen/ verringern

▼ 3. Spiellogik implementieren

3.1 Grenzen

3.1.1 Wand

- wenn der Ball oben/ unten an der Grenze des Spielfelds ankommt soll er abprallen
 - y ist größer/kleiner als die Höhe des Bildschirms
- Abprall
 - Physik: Einfallswinkel= Ausfallswinkel
 - Umsetzung: Gegenteil der aktuellen y-Änderungsrate ($dy = -dy$)
- zu beachten:
 - x,y sind in der Mitte des Balls → Radius noch addieren um an Umriss zu kommen
 - falls Ball über Grenze hinweg, y zurücksetzen → auch hier Radius beachten

3.1.2 Paddles

- Extra/ freiwillig
- funktioniert noch nicht

```
# --- PADDLE 1 ---
    elif ((ball.ycor() + b_radius > p1.ycor() - p_length/2 and
          ball.ycor() - b_radius < p1.ycor() + p_length/2) and
```

```

        (ball.xcor() + b_radius > p1.xcor() - p_widt
h/2 and
        ball.xcor() - b_radius < p1.xcor() + p_widt
h/2)):

        # Compare distance to bottom edge vs top edge
        dist_to_bottom = abs((ball.ycor() + b_radius)
- (p1.ycor() - p_length/2))
        dist_to_top = abs((ball.ycor() - b_radius) -
(p1.ycor() + p_length/2))

        if dist_to_bottom < dist_to_top:
            # Hit bottom edge
            ball.sety(p1.ycor() - p_length/2 - b_radiu
s)
            ball.dy *= -1
        else:
            # Hit top edge
            ball.sety(p1.ycor() + p_length/2 + b_radiu
s)
            ball.dy *= -1

        # --- PADDLE 2 ---
        elif ((ball.ycor() + b_radius > p2.ycor() - p_leng
th/2 and
        ball.ycor() - b_radius < p2.ycor() + p_leng
th/2) and
        (ball.xcor() + b_radius > p2.xcor() - p_widt
h/2 and
        ball.xcor() - b_radius < p2.xcor() + p_widt
h/2)):

        # Compare distance to bottom edge vs top edge
        dist_to_bottom = abs((ball.ycor() + b_radius)
- (p2.ycor() - p_length/2))
        dist_to_top = abs((ball.ycor() - b_radius) -
(p2.ycor() + p_length/2))

```

```

        if dist_to_bottom < dist_to_top:
            # Hit bottom edge
            ball.sety(p2.ycor() - p_length/2 - b_radiu
s)

            ball.dy *= -1
        else:
            # Hit top edge
            ball.sety(p2.ycor() + p_length/2 + b_radiu
s)

            ball.dy *= -1

```

3.2 Treffer

- wenn Ball links/ rechts an Grenzen des Spielfelds kommt
 - x ist größer/kleiner als die Breite des Bildschirms
- nach Treffer:
 - Ball beginnt wieder in der Mitte
 - neue Änderungsrate bestimmen
 - Punktestand hochzählen und aktualisieren

3.3 Abprall vom Paddel

- wenn Ball auf Paddle stößt, soll er abprallen
 - x Koordinate ist größer/kleiner als Grenze des Paddles
 - y Koordinate ist innerhalb der Grenzen des Paddles
 - passiert nur von der Mitte des Spielfeldes aus
- Abprall
 - gleiche Logik wie an den Wänden
 - stattdessen nur x anpassen
- zu beachten:
 - wie eben, Radius dazurechnen
 - dazu auch Breite des Paddles beachten

- wenn Ball im Paddle landet, x zurücksetzen (Radius und Breite beachten)

3.4 Spielende

- bei 10 Punkten → individuell einstellbar
- in jedem Schleifendurchlauf oder bei jeder Punkteerhöhung prüfen
- wenn Punktzahl erreicht, Schleife beenden

▼ 4. schön machen

“Problemstellen”:

- Tastenbewegung → schon vorgeben?
- ausrechnen, wann ein Boundary erreicht ist